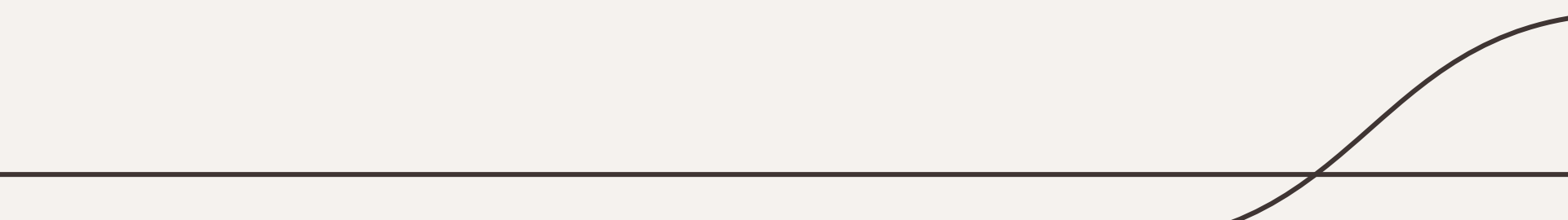


Amazon Project

Team 11



Contents

01

Work Distribution

02

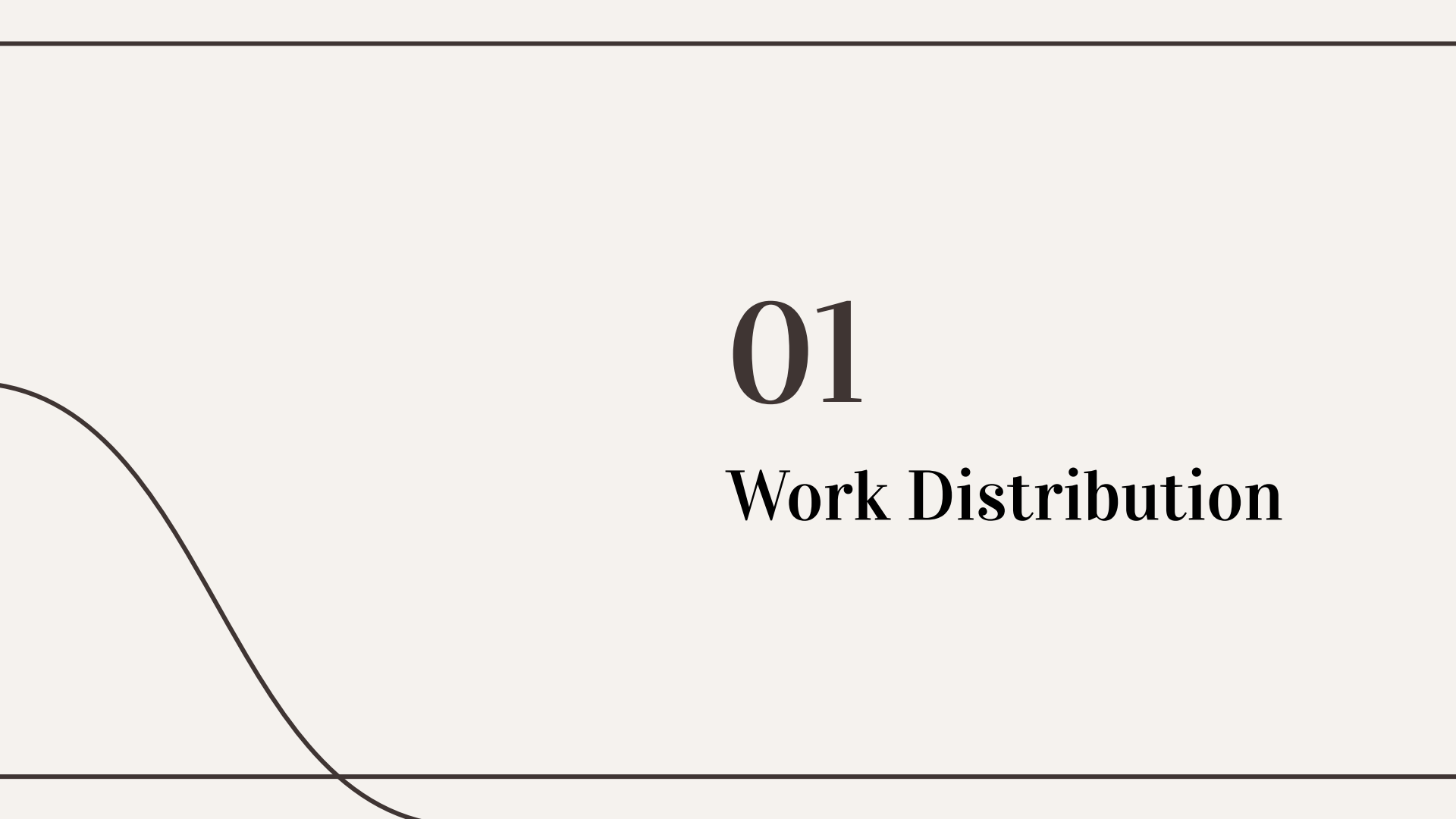
Schema Design

03

Implement Database

04

Query Design

The slide features a light gray background with two horizontal dark gray lines, one at the top and one at the bottom. On the left side, a dark gray curve starts from the top edge and arcs downwards towards the bottom edge.

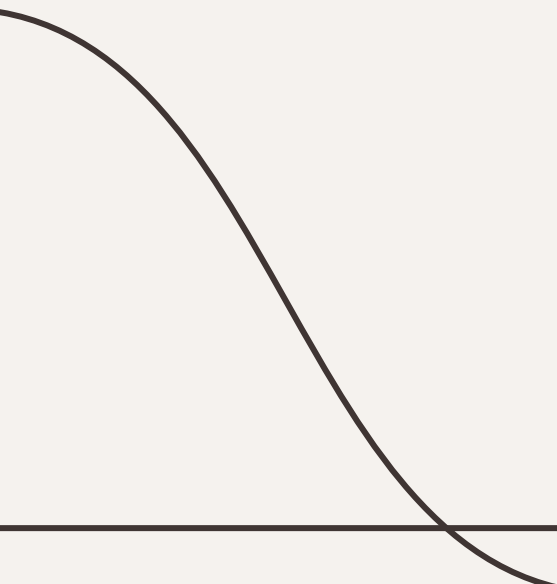
01

Work Distribution

Work Distribution

We separated the project into 3 sub-tasks:

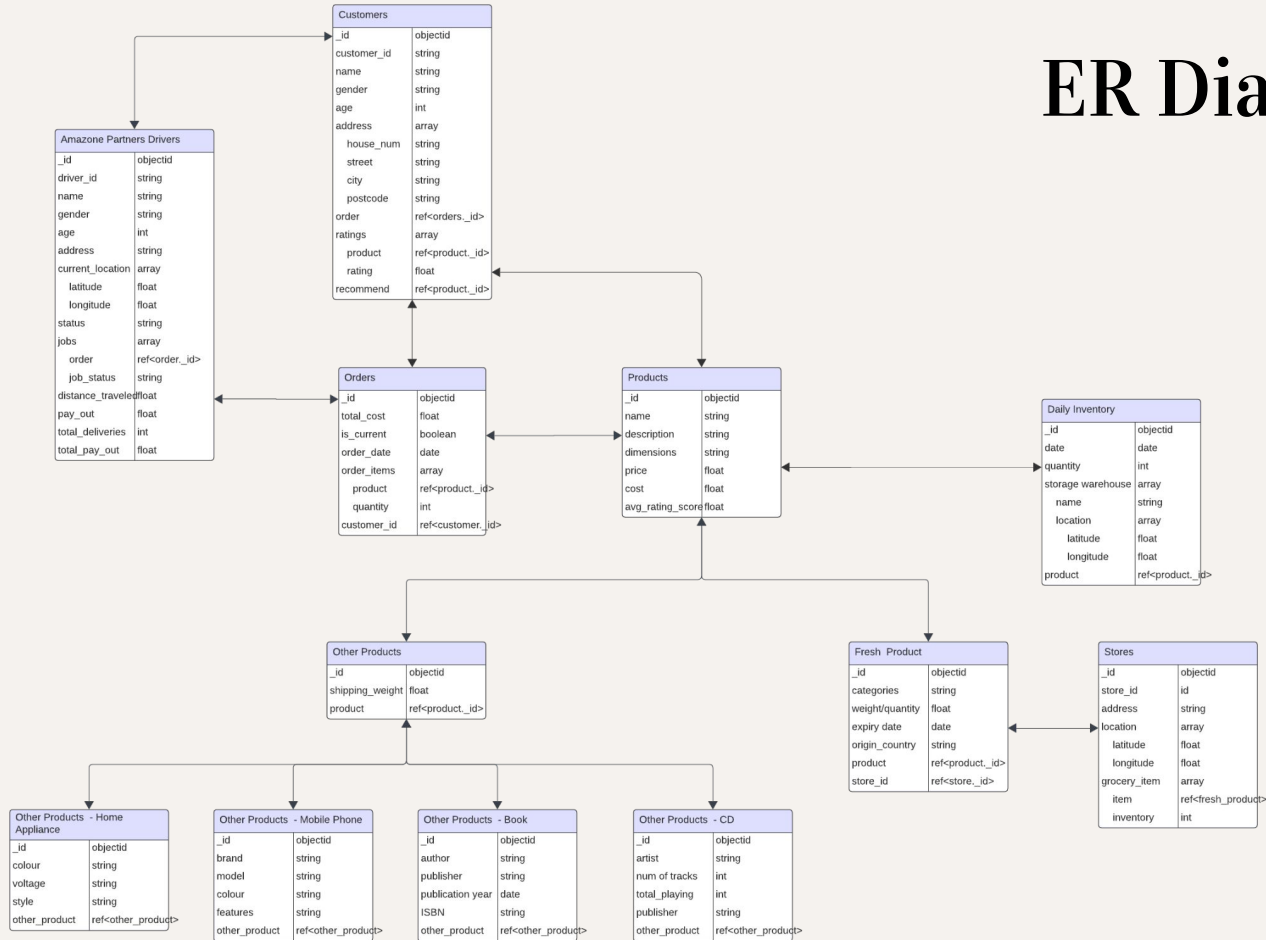
- Task 1: Schema Design: Kuan-Ling Liu, Jahanvi Tripathi
- Task 2: Implement Database: Xinyi Peng, Firaas Nasir
- Tasks 2: Query Design: Jiachen Huang, Michael Tase, Maheen Zahra



02

Schema Design

ER Diagram



Schema Design

```
COLLECTION1 {  
  field1: type                                // for single valued fields  
  field2: {  
    sub-field1: type                          // for embedded fields  
    sub-field2: type  
    ... etc  
  }  
  field3: [ type ]                            // for arrays  
  field4: ref<COLLECTION.Field>              // for referencing fields  
  ...etc  
} // end of COLLECTION1  
  
COLLECTION2 {  
  ... etc  
} // end of COLLECTION2  
  
... etc
```

Schema Design

```
// 1. Customers Collection
customers: {
  _id: ObjectId,
  customer_id: string,
  name: string,
  gender: string,
  age: int,
  address: [{ //embedded address details
    house_number: string,
    street: string,
    city: string,
    postcode: string
  }],
  orders: [ref<orders.order_id>], // reference to orders
  ratings: [{ // ratings by the customer
    product: ref<product._id>, // Reference to product
    rating: float // rating score
  }],
  recommendations: [ref<product._id>] // recommended products
};
```

```
// 2. Products Collection
products: {
  _id: ObjectId,
  name: string, // product name
  description: string, // brief description
  dimensions: string, // dimensions of product
  price: float, // price for customers
  cost: float, // cost to Amazon
  avg_rating_score: float
};
```

```
// 3. Orders Collection
orders: {
  _id: ObjectId,
  total_cost: float, // total cost of the order
  is_current: boolean, // true = current, false = past
  order_date: Date,
  order_items: [{
    product: ref<product._id>, // Reference to product
    quantity: int // qty of product in order
  }],
  customer_id: ref<customer._id>, // customer who placed order
};
```

```
// 4. Fresh Products Collection
fresh_products: {
  _id: ObjectId,
  categories: string, //category(e.g.,bakery,drinks, etc)
  weight_or_quantity: float, // weight or quantity
  expiry_date: date,
  origin_country: string,
  product: ref<product._id>, // reference to product
  store_id: ref<stores._id> // reference to the store
};
```

```
// 5. Other-Products Collection
other_products: {
  _id: ObjectId,
  shipping_weight: float, // shipping weight of product
  product: ref<product._id> // Reference to product
};
```

```
// 6. Other Products - Books Collection
books: {
  _id: ObjectId,
  author: string,
  publisher: string,
  publication_year: Date,
  ISBN: string,
  other_product: ref<other_product._id> //references other product
};
```

```
// 7. Other Products - CDs Collection
cds: {
  _id: ObjectId,
  artist: string,
  number_of_tracks: int,
  total_playing_time: int,
  publisher: string,
  other_product: ref<other_product._id> //references other product
};
```

Schema Design

// 8. Other Products - Mobile Phones Collection

```
mobile_phones: {
  _id: ObjectId,
  brand: string,
  model: string,
  colour: string,
  features: string,
  other_product: ref<other_product._id> //references other product
};
```

// 9. Other Products - Home Appliances Collection

```
home_appliances: {
  _id: ObjectId,
  colour: string,
  voltage: string,
  style: string, // product style
  other_product: ref<other_product._id> //References other product
};
```

// 10. Amazone Partner Stores Collection

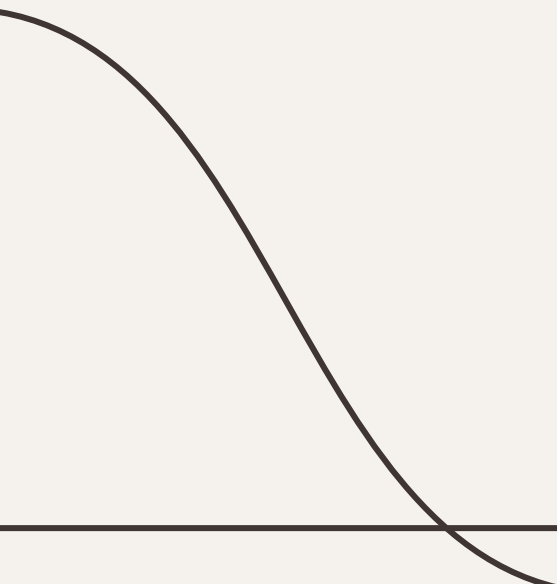
```
stores: {
  _id: ObjectId,
  store_id: string,
  address: string, // store address
  location: { // geolocation of store
    latitude: float, // latitude
    longitude: float // longitude
  },
  grocery_items: [{ // list of grocery items in store
    item: ref<fresh_product._id>,
    inventory: int
  }]
};
```

// 11. Amazone Partner Drivers Collection

```
drivers: {
  _id: ObjectId,
  driver_id: string,
  name: string,
  gender: string,
  age: int,
  address: string,
  current_location: {
    latitude: float,
    longitude: float
  },
  status: string,
  jobs: [{
    order: ref<orders._id>,
    job_status: string, // 'picked', 'in-transit', 'delivered',
    distance_traveled: float, //km
    pay_out: float
  }],
  total_deliveries: int,
  total_pay_out: float,
};
```

// 12. Daily Inventories Collection

```
daily_inventories: {
  _id: ObjectId,
  date: Date, // date of inventory level
  quantity: int, // available quantity
  storage_warehouse: { // storage information
    name: string, // warehouse name
    location: {
      latitude: float,
      longitude: float
    }
  },
  product: ref<product._id> // Reference to Product
};
```



03

Implement Database

Customer recommendations

```
// Fetch all customers
const customers = await customersCollection.find({}).toArray();
console.log(`📦 Fetched ${customers.length} customer(s).`);

// Fetch all products
const products = await productsCollection.find({}).toArray();
console.log(`📦 Fetched ${products.length} product(s).`);
```

```
for (let customer of customers) {
  try {
    // Fetch products the customer has purchased (current and past orders)
    const customerOrders = await ordersCollection.find({ customer_id: customer._id }).toArray();
    const purchasedProductIds = new Set();
    customerOrders.forEach(order => {
      if (order.order_items && Array.isArray(order.order_items)) {
        order.order_items.forEach(item => {
          if (item.product) {
            purchasedProductIds.add(item.product.toString());
          }
        });
      }
    });

    // Fetch products the customer has rated
    const ratedProductIds = new Set();
    if (customer.ratings && Array.isArray(customer.ratings)) {
      customer.ratings.forEach(rating => {
        if (rating.product) {
          ratedProductIds.add(rating.product.toString());
        }
      });
    }
  }
}
```

```
// Identify eligible products for recommendation (not purchased or rated)
let eligibleProducts = products.filter(prod => !excludedProductIds.has(prod._id.toString()));
let recommendations = [];
if (eligibleProducts.length >= RECOMMENDATIONS_PER_CUSTOMER) {
  // Shuffle eligible products
  eligibleProducts = shuffleArray(eligibleProducts);
}
```

```
if (eligibleProducts.length < RECOMMENDATIONS_PER_CUSTOMER) {
  const additionalEligibleProducts = products.filter(prod => !recommendations.includes(prod._id));
  const shuffledAllProducts = shuffleArray(products);
  recommendations = shuffledAllProducts.slice(0, RECOMMENDATIONS_PER_CUSTOMER).map(prod => prod._id);
}
```

Ratings

```
const aggregationPipeline = [
  { $unwind: "$ratings" },
  {
    $group: {
      _id: "$ratings.product",
      averageRating: { $avg: "$ratings.rating" },
      count: { $sum: 1 }
    }
  }
];
const aggregationResults = await customersCollection.aggregate(aggregationPipeline).toArray();
```

```
const bulkOperations = aggregationResults.map(result => ({
  updateOne: {
    filter: { original_id: result._id },
    update: { $set: { avg_rating_score: parseFloat(result.averageRating.toFixed(2)) } }
  }
}));
const bulkWriteResult = await productsCollection.bulkWrite(bulkOperations);
```

```
const productsWithNoRatings = await productsCollection.find({
  original_id: { $nin: ratedOriginalIds }
}).toArray();
const bulkOperationsNoRatings = productsWithNoRatings.map(product => ({
  updateOne: {
    filter: { _id: product._id },
    update: { $set: { avg_rating_score: 0 } }
  }
}));
await productsCollection.bulkWrite(bulkOperationsNoRatings);
```

Inventory

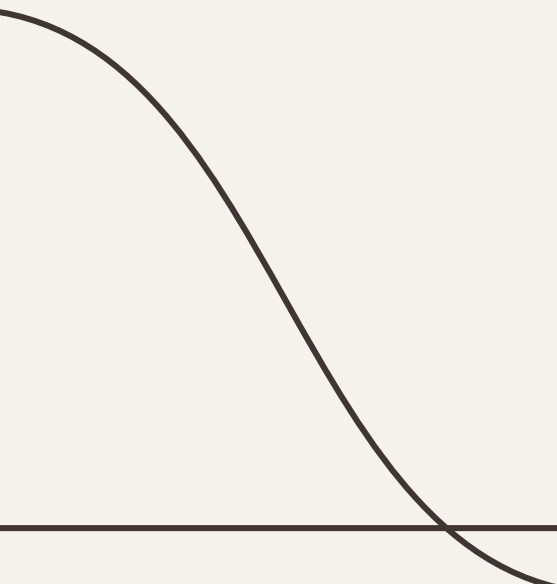
```
function assignWarehouse() {  
  const usePredefined = Math.random() < 0.7;  
  if (usePredefined) {  
    const randomIndex = Math.floor(Math.random() * predefinedWarehouses.length);  
    return predefinedWarehouses[randomIndex];  
  } else {  
    const randomCoords = generateRandomCoordinates();  
    return {  
      name: "Random Warehouse",  
      location: randomCoords  
    };  
  }  
}
```

```
const inventoryLog = {  
  date: logDate,  
  quantity: quantity,  
  storage_warehouse: storageWarehouse,  
  product: product._id  
};  
await dailyInventoriesCollection.insertOne(inventoryLog);
```

Database Index Overview

Collection	Additional Indexes	Index Reference Description
Customers	orders_1	References orders._id
	recommendations_1	References product._id
	product_1	References product._id
Orders	order_items.products_1	References product._id
	customer_id_1	References Customers._id
Fresh_products	products_1	References product._id
	store_id_1	References Partner_Stores._id
other_products	product_idx	References product._id
Other_Products_Books	other_product_1	References other_products._id
Other_Products_CDs	other_product_1	References other_products._id
Other_Products_MobilePhones	other_product_1	References other_products._id
Other_Products_HomeAppliances	other_product_1	References other_products._id
Partner_Stores	grocery_items.item_1	References Fresh_products._id
	jobs.order_1	References orders._id
Partner_Drivers	current_location_2dsphere	2dsphere for geo queries
daily_inventories	product_date_idx	References product._id

Note: The default index, which is automatically created on the _id field, is ignored in the table.



04

Query Design

Task 1. Customer ordering a fresh product.

- Identify a customer & a fresh product to be ordered via IDs.
- **Q1:** Aggregate fresh_products collection to products using *\$match*, *\$lookup*, and *\$unwind* and get the desired product document with store details.
- Create a 'current_location' index for efficient calculation of distances.
- **Q2:** Aggregate drivers collection using *\$geoNear* operator getting distances relative to store location
- Select the first driver from the list with the smallest distance.
- **Q3:** Create an order and use *insert_one* to save it.
- **Q4:** Use *update_one* to add order on customer document.
- **Q5:** Create a job/task for the driver and use *update_one* to update the driver's document.

Task 1. Customer ordering a fresh product. (Final Result)

```
{'driver_assigned': {'distance_to_store': 147.95461186475927,  
                     'location': {'coordinates': [-0.2014, 51.4101],  
                                   'type': 'Point'},  
                     'name': 'John Doe',  
                     'pay_out': 0.349000000000000003},  
 'product_ordered': {'category': 'bakery',  
                     'expiry_date': datetime.datetime(2024, 12, 5, 0, 0),  
                     'name': 'Cinnamon Roll',  
                     'price': 3.49,  
                     'weight_or_quantity': 200}}
```

The above shows the Driver assigned the delivery job on the product.

Task 2. Customer searching for available fresh products close to them.

- **Q1:** Use *find_one* to get the customer's document.
- **Q2:** Use *find* to get nearby stores based on the customer's city in 'address'.
- **Q3:** Aggregate *fresh_products* collection to products using *\$match* & *\$in* operators to get products with *store_id* matching nearby store ids.
- List the products into a data frame.

	_id	expiry_date	store_id	name	price	category
0	6752d39b9cab0e03823a19b1	2024-12-07	6752d4f69cab0e03823a19bb	Carrot	0.89	fruits and vegetables
1	6752d39b9cab0e03823a19a7	2024-12-07	6752d4f69cab0e03823a19bb	Chocolate Eclair	3.49	bakery
2	6752d39b9cab0e03823a19ac	2024-12-18	6752d4f69cab0e03823a19bb	Fresh Orange Juice	2.49	drinks

Task 3. Customer ordering a product, adding it to the cart, and making payment.

- **Q1:** Use *find_one* to get the customer's record.
- Identify the 'book' products to be ordered via their IDs.
- **Q2:** Aggregate 'books' collection to 'other_products' and then to full book products using *\$match*, *\$in*, *\$lookup*, and *\$unwind*.
- **Q3:** Create an order and use *insert_one* to save it. (Creates an order in the cart.)
- At this stage, the order is in the cart as the '*is_current*' property shows '**True**' signifying a current order.
- **Q4:** Use *update_one* to update order in cart using *\$set* by making '*is_current*': **False**, signifying a purchased order and making it a past order.
- **Q5:** Use *update_one* to add order to customer's orders document.

Task 3. Customer ordering a product, adding it to the cart, and making payment.

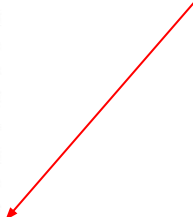
Customer Record with final Result of Queries

Books to be ordered

```
[{'_id': ObjectId('6752ccaefa582a35118e3626'),
  'name': 'The Great Gatsby',
  'price': 11.99,
  'product_id': ObjectId('6754b416318d88a8ccb44232'),
  'qty': 3},
 {'_id': ObjectId('6752ccaefa582a35118e3628'),
  'name': 'Pride and Prejudice',
  'price': 9.99,
  'product_id': ObjectId('6754b416318d88a8ccb44234'),
  'qty': 1}]
```

```
{'_id': ObjectId('64e1fab1234567890abcd007'),
  'address': {'city': 'Glasgow',
              'house_number': 321,
              'postcode': 'G1 3PQ',
              'street': 'Queens Road'},
  'age': 27,
  'customer_id': 'CUST007',
  'gender': 'Female',
  'name': 'Grace Hall',
  'orders': [ObjectId('6752e00e9cab0e03823a1a4f'),
             ObjectId('6752e00e9cab0e03823a1a50'),
             ObjectId('6752e00e9cab0e03823a1a51'),
             ObjectId('6752e00e9cab0e03823a1a52'),
             ObjectId('6752e00e9cab0e03823a1a53'),
             ObjectId('6752e00e9cab0e03823a1a54'),
             ObjectId('6752e00e9cab0e03823a1a55'),
             ObjectId('675b33e0c045102a2fb89a0f'),
             ObjectId('675c46b1c045102a2fb89a12')],
  'ratings': [{'product': ObjectId('6754b399f6dd978e8741333c'), 'rating': 4},
               {'product': ObjectId('6754b399f6dd978e8741333d'), 'rating': 3.4},
               {'product': ObjectId('6754b399f6dd978e8741333e'), 'rating': 3.4}],
  'recommendations': [ObjectId('6754b399f6dd978e8741333b'),
                      ObjectId('6754b399f6dd978e8741334f')]}
```

Latest Order Added

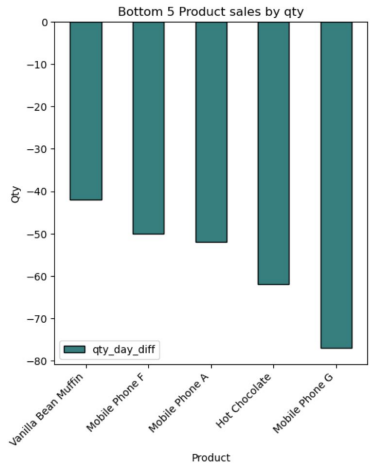
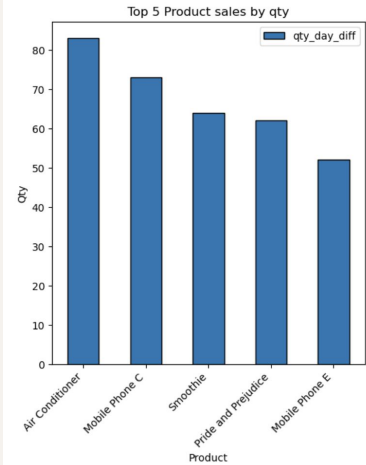
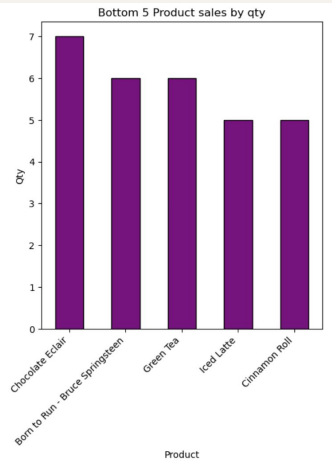
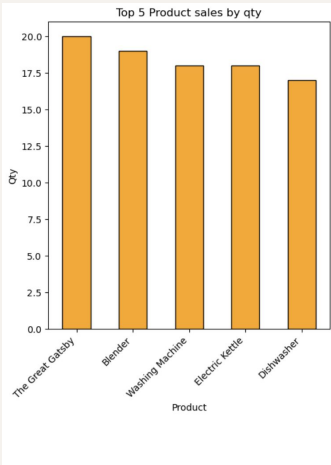
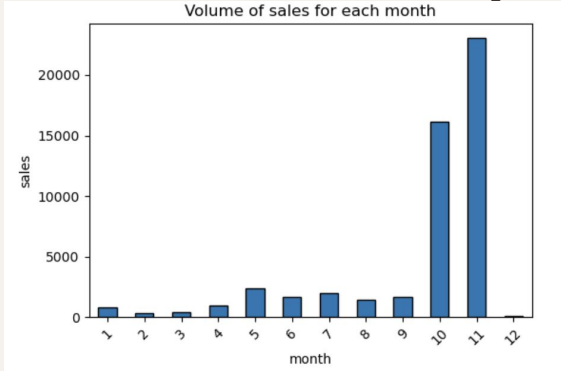
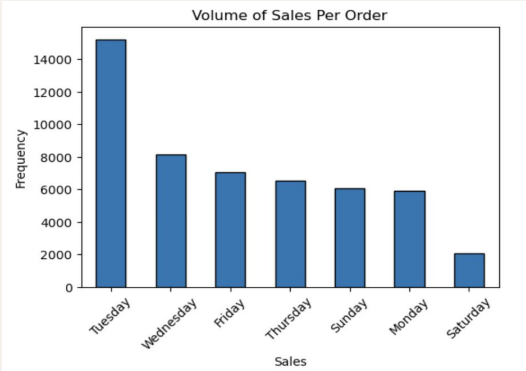
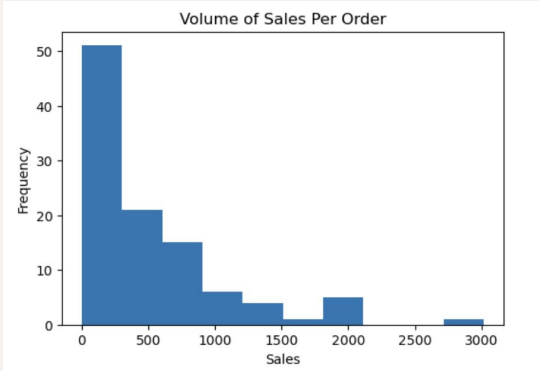


Above shows the user ordering a book and payment making the order complete

Task 4. Checking sales and inventory performance.

- **Q1:** Select past orders (completed sales) using *\$match* where *is_current* is **False**.
- Read the result into a Pandas data frame.
- Plot volume of sales to show distribution by the orders.
- Plot sales per day of the week and month of the year.
- **Q2:** Select 'orders' collection and aggregate order items using *\$unwind*, *\$lookup*, to select product details in each order item.
- Read all order items into a Pandas Data frame
- Plot number of product sales by quantity
- **Q3:** Select inventories for the two days, 6th and 7th December, and aggregate to product details.
- Read results into Panda's data frame and merge into one on product name.
- Take the difference in quantity for each product for the two days and plot the results to show how the stock rises or falls.

Task 4. Checking sales and inventory performance. (Plots)



Task 5. Query for Sales Performance Analysis:

Analyze the total sales revenue of different product categories and sort them by revenue.

```
# Classify the product category
{
  "$addFields": {
    product_category": {
      "$cond": [
        { "$gt": [{ "$size": "$fresh_product_details" }, 0] },
        { "$concat": ["Fresh Products - ", { "$arrayElemAt": [ "$fresh_product_details.categories", 0 ] } ] },
        {
          "$cond": [
            { "$gt": [{ "$size": "$book_details" }, 0] },
            "Other Products - Books",
            {
              "$cond": [
                { "$gt": [{ "$size": "$cd_details" }, 0] },
                "Other Products - CDs",
                {
                  "$cond": [
                    { "$gt": [{ "$size": "$home_appliance_details" }, 0] },
                    "Other Products - Home Appliances",
                    {
                      "$cond": [
                        { "$gt": [{ "$size": "$mobile_phone_details" }, 0] },
                        "Other Products - Mobile Phones",
                        "Unknown"
                      ]
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  }
}
```

_id	total_sales	total_quantity
Other Products - Home Appliances	10229.41	59
Other Products - Mobile Phones	8449.85	15
Other Products - Books	2409.69	181
Other Products - CDs	2382.37	165
Fresh Products - drinks	365.99	101
Fresh Products - bakery	130.86	44
Fresh Products - fruits and vegetables	111.65	55

_id	total_quantity	total_revenue
Washing Machine	2	999.98
Air Conditioner	1	349.99
Microwave Oven	2	199.98
Electric Kettle	3	89.97
Blender	1	49.99
The Great Gatsby	3	35.97
The Catcher in the Rye	2	21.98
The Hobbit	2	20.98
Bridge Over Troubled Water - Simon & Garfunkel	1	12.49
Pride and Prejudice	1	9.99

A customer ordering **Other Products** with sales quantity and total revenue calculated for each product.

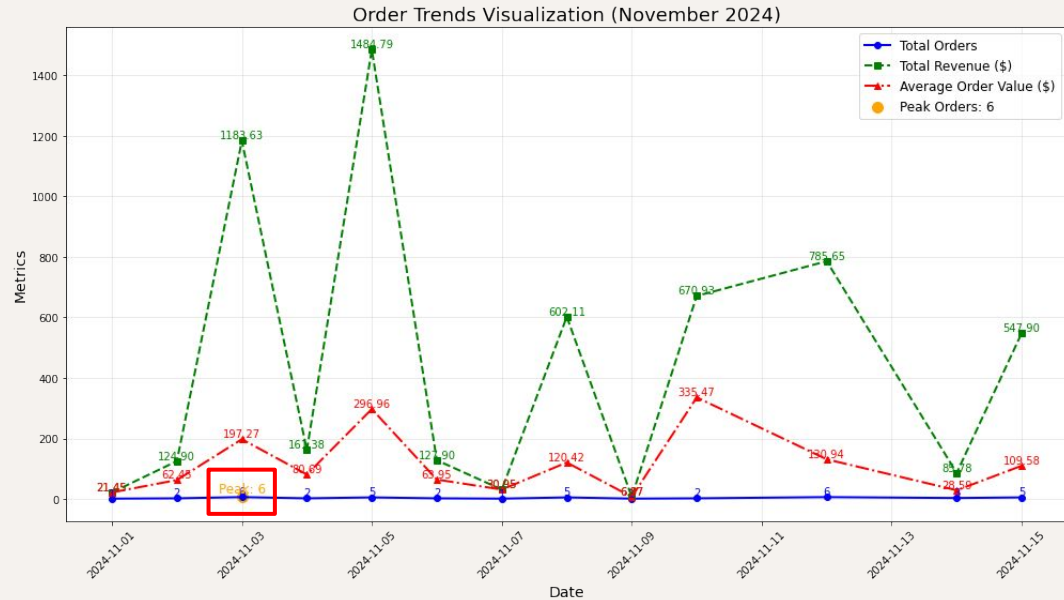
```
# Define the aggregation pipeline
pipeline = [
    # Match orders for a specific customer
    {
        "$match": {"customer_id": ObjectId("64e1fab1234567890abcd005")}
    },

    # Unwind the order_items array to process each item individually
    {"$unwind": "$order_items"},

    # Lookup product details from the products collection
    {
        "$lookup": {
            "from": "products",
            "localField": "order_items.product",
            "foreignField": "_id",
            "as": "product_details"
        }
    },
    {"$unwind": "$product_details"},
]
```

Query Order Trends (total orders, total revenue, and average order value)

```
# MongoDB aggregation pipeline
pipeline = [
  {
    "$match": {
      "order_date": {
        "$gte": datetime(2024, 11, 1, 0, 0),
        "$lte": datetime(2024, 11, 15, 23, 59, 59)
      }
    },
  },
  {
    "$group": {
      "_id": { "$dateToString": { "format": "%Y-%m-%d", "date": "$order_date" } },
      "total_orders": { "$sum": 1 },
      "total_revenue": { "$sum": "$total_cost" },
      "average_order_value": { "$avg": "$total_cost" }
    },
  },
  {
    "$sort": { "_id": 1 },
  },
  {
    "$project": {
      "date": "$_id",
      "total_orders": 1,
      "total_revenue": 1,
      "average_order_value": 1,
      "_id": 0
    },
  },
]
```



The process dynamically calculates **product ratings**, identifies **products not purchased** by each customer, and **recommends the top two highest-rated products** based on average scores.

avg_ratings

```
{'6754b416318d88a8ccb44239': 4.75,  
'6752d39b9cab0e03823a19b3': 3.8,  
'6752cee39cab0e03823a1989': 3.1,  
'6754b399f6dd978e8741334d': 3.3666666666666667,  
'6754b416318d88a8ccb4423b': 3.65,  
'6754b416318d88a8ccb4423c': 4.15,  
'6754b399f6dd978e87413341': 3.45,  
'6754b399f6dd978e87413344': 3.45,  
'6754b399f6dd978e87413356': 4.033333333333333,  
'6752cf7f9cab0e03823a198e': 4.0,  
'6754b416318d88a8ccb4423a': 3.8,  
'6754b399f6dd978e87413352': 4.383333333333334,  
'6752d39b9cab0e03823a19aa': 3.75,  
'6754b399f6dd978e8741333c': 4.233333333333333,  
'6754b416318d88a8ccb44241': 4.1,  
'6754b416318d88a8ccb44230': 4.0600000000000005,
```

```
'64e1fab1234567890abcd016': [{'_id': ObjectId('6754b416318d88a8ccb44239'),  
  'avg_rating_score': 4.75,  
  'name': 'Abbey Road - The Beatles'},  
  {'_id': ObjectId('6754b398f6dd978e87413335'),  
  'avg_rating_score': 4.475,  
  'name': 'Classic Croissant'}],  
'64e1fab1234567890abcd017': [{'_id': ObjectId('6754b399f6dd978e8741334a'),  
  'avg_rating_score': 4.766666666666667,  
  'name': 'Dishwasher'},  
  {'_id': ObjectId('6754b399f6dd978e87413347'),  
  'avg_rating_score': 4.6000000000000005,  
  'name': 'Air Conditioner'}],
```

The image features two thin, dark horizontal lines. The top line starts with a curved segment on the left side, and the bottom line ends with a curved segment on the right side.

Thanks